UNIVERSIDAD PERUANA CAYETANO HEREDIA
FACULTAD DE CIENCIAS E INGENIERÍA
E.A.P Ingeniería Informática



MISIÓN EN MARTE

Informe de Proyecto Parcial

Curso:

Programación Orientada a Objetos

Profesores:

MSc. Wilder Nina Choquehuayta

Dr. Emanuel Ortiz Ortiz

Integrantes:

Bances Panaque, Jose Junior

Barzola Reategui, Josué Obed

Cerdan Dongo, Rebeca Milagros

Salcedo Siguas, Yanella Stefhany

Lima, octubre de 2026

I. Resumen Ejecutivo

Este informe presenta el avance parcial de «Misión en Marte», un videojuego 2D desarrollado en Python y Pygame cuyo objetivo es demostrar el dominio de la Programación Orientada a Objetos. La arquitectura se organiza en 18 clases alrededor de la jerarquía abstracta EntidadJuego, con un Game Loop que invoca update() de forma polimórfica. Se aplican asociación, agregación, composición, encapsulamiento, herencia y polimorfismo, junto con los patrones Factory Method (TormentaFactory) y State (Marciano). El avance incluye tres niveles jugables.

II. Introducción

a. Contexto y objetivo

El presente informe documenta el avance parcial del proyecto final del curso de Programación Orientada a Objetos (ciclo 2026-2). El proyecto es una evaluación integral: el equipo debe construir un videojuego en el que se evidencie, tanto en el diseño como en el código, el uso correcto de los conceptos vistos en las sesiones teóricas.

«Misión en Marte» es un videojuego 2D de acción y supervivencia. El jugador controla a un **Explorador** que debe recolectar combustible, construir bases de refugio, ganarse la confianza de marcianos pacíficos y sobrevivir a tormentas de radiación, lluvias de meteoritos y criaturas hostiles, hasta escapar del planeta en un cohete.

Objetivo principal: demostrar el dominio de la POO (herencia, encapsulamiento, polimorfismo y composición) y de los patrones de diseño Factory Method y State, mediante una arquitectura coherente con el diagrama UML y un juego jugable.

Objetivos específicos:

- Diseñar un diagrama de clases UML con más de 8 clases acordes al juego.
- Implementar y explicar los seis pilares: asociación, agregación, composición, encapsulamiento, herencia y polimorfismo.
- Entregar un avance jugable con interfaces y funcionalidades relevantes.
- Versionar el código con Git de forma colaborativa.

b. Tecnologías utilizadas

Tecnología	Uso en el proyecto
Python 3	Lenguaje de implementación. Módulos estándar abc, typing, math y random.
Pygame 2.x	Ventana gráfica, Game Loop, lectura de teclado y dibujo 2D.
Visual Studio Code	Editor y entorno de ejecución del juego.
draw.io y Graphviz	Elaboración del diagrama de clases UML.
Git	Versionamiento en equipo. Plataforma: GitHub

III. Diseño de Arquitectura

a. Diagrama de clases (UML)

La Figura 1 (página siguiente) presenta el diagrama de clases del juego. Contiene **18 clases**, entre ellas tres clases abstractas (EntidadJuego, Nivel y Tormenta) y una interfaz (EstadoMarciano), resaltadas en azul. El diagrama muestra relaciones de herencia, composición, agregación, asociación y dependencia, resumidas en la Tabla 1.

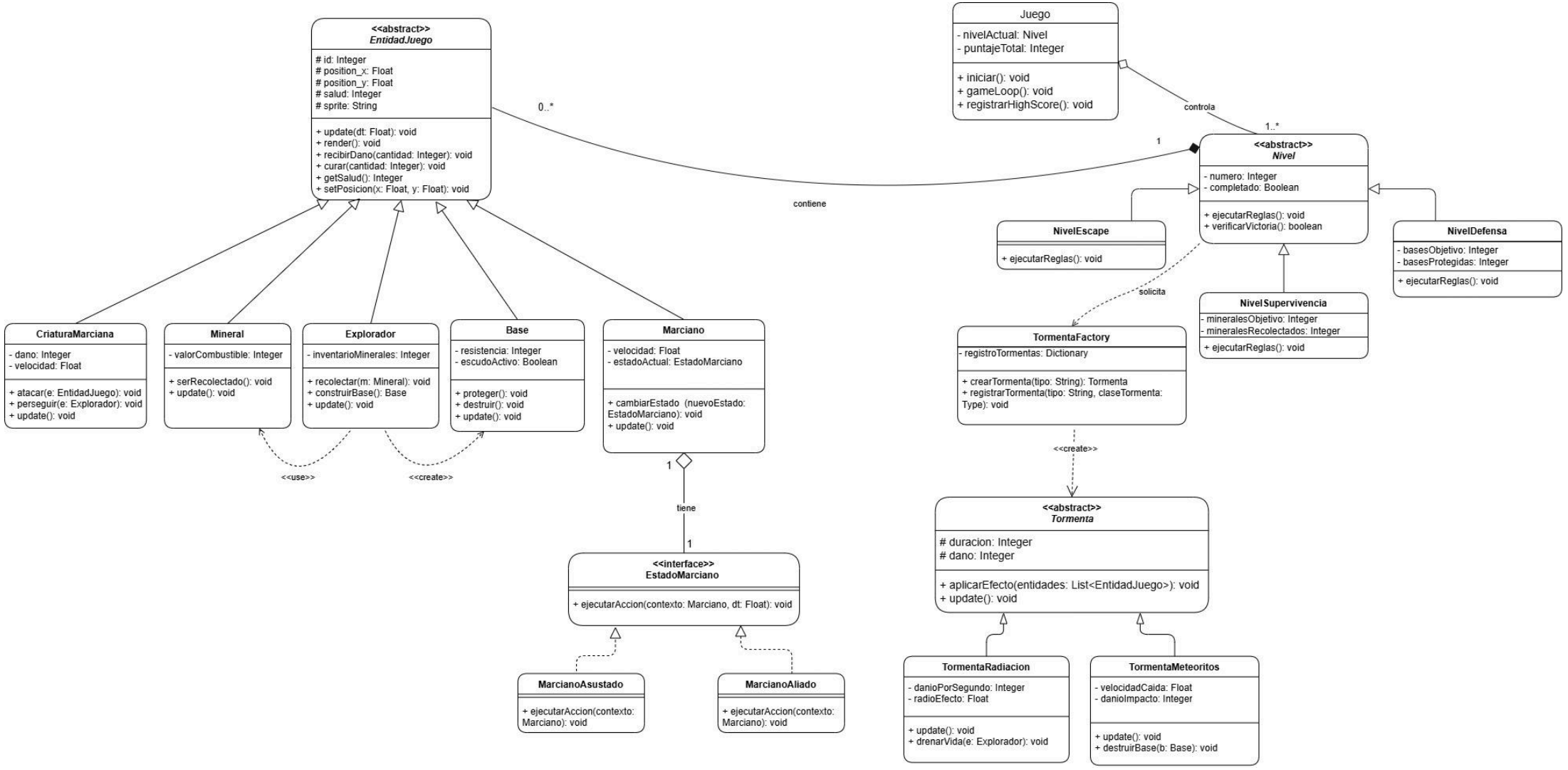


Figura 1. Diagrama de clases UML de «Misión en Marte».

Tabla 1. Relaciones del diagrama de clases

Relación	Clases	Justificación en el juego
Herencia	EntidadJuego ← Mineral, Explorador, Base, Marciano, CriaturaMarciana	Todas las entidades del mapa comparten identidad, posición, salud y el contrato update() / render().
Herencia	Nivel ← NivelSupervivencia, NivelDefensa, NivelEscape	Cada nivel redefine sus reglas, su objetivo y su condición de victoria.
Herencia	Tormenta ← TormentaRadiacion, TormentaMeteoritos	Cada tormenta implementa su propio efecto sobre las entidades.
Realización de interfaz	EstadoMarciano ← MarcianoAsustado, MarcianoAliado	Cada estado define la acción del marciano (patrón State).
Composición	Nivel ◆ EntidadJuego; Nivel ◆ Tormenta	El nivel crea sus entidades y tormentas; desaparecen con él.
Agregación	Juego ◇ Nivel	Los niveles se inyectan al juego y tienen vida propia.
Agregación	Marciano ◇ EstadoMarciano	El estado se inyecta desde fuera y puede cambiarse en ejecución.
Asociación	Base → Explorador; CriaturaMarciana → Explorador	La base conoce a su dueño; la criatura conoce a quién persigue.
Dependencia	Explorador ⇢ Mineral «use»; Explorador ⇢ Base «create»; Nivel ⇢ TormentaFactory «solicita»; TormentaFactory ⇢ Tormenta «create»	Relaciones temporales: se usan o se crean durante una operación, sin ser atributos permanentes.

b. Estructura de componentes

Las clases principales y el rol que cumplen dentro del juego son:

Tabla 2. Componentes principales

Clase	Rol
`Juego`	Controla el Game Loop (gameLoop), las fases (menú, jugando, nivel completo, derrota y final), el HUD y el puntaje.
`Nivel y subclases`	Contiene entidades y tormentas, las actualiza polimórficamente y define objetivo y victoria de cada nivel.
`EntidadJuego`	Clase base abstracta: posición, salud encapsulada y contrato update() / render().

Clase	Rol
`Explorador`	Personaje del jugador: se mueve, recolecta minerales y construye bases.
`Mineral`	Combustible del mapa; no es afectado por las tormentas.
`Base`	Refugio con escudo que absorbe daño y cura al explorador cercano.
`Marciano` y `EstadoMarciano`	Habitante pacífico. Delega su conducta en un estado (asustado o aliado).
`CriaturaMarciana`	Enemigo que persigue y ataca al explorador.
`Tormenta` y subclases	Eventos ambientales: zona de radiación y lluvia de meteoritos.
`TormentaFactory`	Fábrica que crea tormentas a partir de un identificador registrado.

El código se organiza en módulos que reflejan esa estructura:

Archivo	Contenido
`main.py`	Punto de entrada: registra las tormentas en la fábrica, crea los niveles e inicia el juego.
`src/juego.py`	Clase Juego (Game Loop, HUD, fases y puntaje).
`src/niveles/__init__.py`	Clases de niveles (Nivel, NivelSupervivencia, NivelDefensa, NivelEscape).
`src/entidades/__init__.py`	Jerarquía de entidades (EntidadJuego, Mineral, Explorador, Base, Marciano, CriaturaMarciana) y Patrón State (EstadoMarciano, etc.).
`src/tormentas/__init__.py`	Clases ambientales (Tormenta, TormentaRadiacion, TormentaMeteoritos) y TormentaFactory.
`src/config.py`, `src/graficos.py`	Constantes globales del juego y utilidades de renderizado alfa.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación

Es una relación de conocimiento entre objetos independientes: uno guarda una referencia al otro para poder colaborar. En el juego, cada Base conoce a su dueño (el Explorador) para curarlo mientras está dentro de su zona de refugio, y cada Marciano conoce al objetivo del que huye o al que sigue.

```
# Base.__init__ y Marciano.__init__ (asociación)
    def __init__(self, x: float, y: float, dueno: Explorador,
    resistencia: int = 120) -> None:
        super().__init__(x, y, resistencia, "base", 26)
        self.__dueno: Explorador = dueno          # ASOCIACIÓN: la base
    conoce a su dueño
        self.__resistencia: int = resistencia
        self.__escudoActivo: bool = True
        self.__energiaEscudo: int = 40
        self.__recarga: float = 0.0
        self.__curaAcumulada: float = 0.0
    def __init__(self, x: float, y: float, objetivo: EntidadJuego,
                 estadoInicial: EstadoMarciano) -> None:
        super().__init__(x, y, 40, "marciano", 14)
        self.__velocidad: float = 90.0
        self.__objetivo: EntidadJuego = objetivo
        self.__estadoActual: EstadoMarciano = estadoInicial   #
    AGREGACIÓN: estado inyectado
        self.__rumbo: float = random.uniform(0, 2 * math.pi)
        self.__cambioRumbo: float = 0.0
        self.__curaAcumulada: float = 0.0
```

Código 1. Asociación: `Base` → `Explorador` y `Marciano` → objetivo.

Agregación

Es una relación «tiene un» débil: el objeto contenido nace fuera y sobrevive a quien lo usa. Juego recibe sus niveles ya construidos; Marciano recibe su estado inicial desde el exterior; y cuando el Explorador recolecta un Mineral, la destrucción del explorador no implica la del mineral del ecosistema.

```
    # Juego.__init__ (fragmento)
    # AGREGACIÓN: los niveles se inyectan desde fuera y tienen vida
    propia
    self.__niveles: List[Nivel] = niveles

    def recolectar(self, m: Mineral) -> None:
        """AGREGACIÓN/USO: el mineral existe en el ecosistema, el
    explorador solo lo toma."""
        valor = m.getValorCombustible()
        m.serRecolectado()
        self.almacenar(valor)
```

Código 2. Agregación: niveles inyectados en `Juego` y mineral tomado por el `Explorador`.

Composición

Es la relación «parte de» fuerte: las partes no existen sin el todo. Nivel crea sus propias entidades (minerales, marcianos, criaturas, bases) y tormentas; nadie más las instancia. Al

reiniciar el nivel se vacían todas las listas y, durante el juego, __limpiarMuertos() elimina las referencias a entidades muertas para evitar fugas lógicas de memoria.

```
    def reiniciar(self) -> None:
        self.__entidades.clear()
        self.__tormentas.clear()
        self._minerales.clear()
        self._bases.clear()
        self._marcianos.clear()
        self._criaturas.clear()
        self.__completado = False
        self._tiempo = 0.0
        self.__explorador = Explorador(config.ANCHO / 2,
    config.ALTO / 2 + 20)
        self.__entidades.append(self.__explorador)
        self._poblar()

    def _crearMineral(self) -> Mineral:
        x, y = self._posicionLibre(60)
        m = Mineral(x, y, random.choice([1, 1, 1, 2]))
        self.__entidades.append(m)
        self._minerales.append(m)
        return m
```

Código 3. Composición: `Nivel` crea y destruye sus entidades.

Además, el Explorador mantiene una dependencia de creación con Base: construye una base solo si tiene los minerales necesarios.

```
    def construirBase(self) -> Optional[Base]:
        """DEPENDENCIA <<create>>: crea una Base si alcanzan los
    minerales."""
        if not self.gastarMinerales(self.COSTO_BASE):
            return None
        return Base(self._position_x, self._position_y, self)
```

Código 4. Dependencia «create»: `Explorador.construirBase()`.

b. Herencia y polimorfismo

Toda entidad interactiva hereda de la clase abstracta EntidadJuego, que define el contrato común. La jerarquía de los elementos del juego es:

- EntidadJuego (abstracta)
  - Mineral, Explorador, Base, Marciano, CriaturaMarciana
- Nivel (abstracta): NivelSupervivencia, NivelDefensa, NivelEscape
- Tormenta (abstracta): TormentaRadiacion, TormentaMeteoritos
- EstadoMarciano (interfaz): MarcianoAsustado, MarcianoAliado

El Game Loop recorre una colección genérica de EntidadJuego y llama a update() sin distinguir entre los tipos de objeto; cada subclase ejecuta su propia versión. Lo mismo ocurre con las tormentas. En ningún punto del código se usa isinstance() ni se compara un atributo de tipo para decidir un comportamiento.

```
    def update(self, dt: float) -> None:
        self._tiempo += dt
        for entidad in self.__entidades:          # POLIMORFISMO: no
    importa el tipo concreto
            entidad.update(dt)
        self.__procesarColisiones()
        for tormenta in self.__tormentas:         # POLIMORFISMO también
    en tormentas
            tormenta.update(dt)
            tormenta.aplicarEfecto(self.__entidades)
        self.__tormentas = [t for t in self.__tormentas if not
    t.haTerminado()]
        self.ejecutarReglas(dt)
        self.__limpiarMuertos()
        if self.verificarVictoria():
            self.__completado = True
```

Código 5. Llamada polimórfica a `update()` y `aplicarEfecto()` dentro de `Nivel.update()`.

Para que las tormentas no tengan que preguntar «¿qué eres?», se usa un método gancho polimórfico: por defecto toda entidad es afectable y Mineral lo sobrescribe. Además, las subclases reutilizan la lógica base con super().

```
    # EntidadJuego
    def esAfectablePorTormenta(self) -> bool:
        """Hook polimórfico: evita preguntar el tipo de la entidad."""
        return True

    # Mineral
    def esAfectablePorTormenta(self) -> bool:
        return False

    # Marciano
    def update(self, dt: float) -> None:
        super().update(dt)
        self.__estadoActual.ejecutarAccion(self, dt)   # delegación al
    estado
```

Código 6. Sobrescritura de métodos y uso de `super()`.

c. Encapsulamiento y abstracción

Los atributos críticos son privados (name mangling con doble guion bajo), de modo que solo la propia clase puede mutarlos. No existe un setSalud(): el estado cambia únicamente mediante métodos de comportamiento con validación. Las subclases acceden a la información mediante getters.

Tabla 3. Atributos críticos protegidos

Atributo	Clase	Justificación y forma de acceso
`__salud`	`EntidadJuego`	Un valor fuera de rango rompería el juego. Se lee con getSalud() y solo cambia con recibirDano() y curar().
`__inventarioMinerales`	`Explorador`	Recurso que se gasta al construir. Se accede con getInventarioMinerales(), almacenar() y gastarMinerales().
`__estadoActual`	`Marciano`	Solo debe cambiar por reglas del juego: cambiarEstado() y ofrecerMineral().
`__escudoActivo, __energiaEscudo`	`Base`	El escudo se gestiona internamente: proteger() lo activa y recibirDano() lo desgasta.
`__registroTormentas`	`TormentaFactory`	Evita que se modifique el registro sin pasar por registrarTormenta().
`_position_x, _position_y`	`EntidadJuego`	Protegidos (_). Se modifican con setPosicion(), que valida los límites del mapa.

Sobre los setters: solo se expone setPosicion(), porque valida que la entidad no salga de la pantalla. Los datos vitales se modifican con operaciones del dominio (recibirDano, curar) que validan sus parámetros.

```
    # EntidadJuego.__init__ (fragmento)
    # Atributos críticos: privados (encapsulamiento estricto)
    self.__salud: int = salud
    self.__saludMaxima: int = salud

    def recibirDano(self, cantidad: int) -> None:
```

```
        if cantidad < 0:
            raise ValueError("el daño no puede ser negativo")
        if not self.estaViva():
            return
        self.__salud = max(0, self.__salud - cantidad)
        if self.__salud == 0:
            self._alMorir()

    def curar(self, cantidad: int) -> None:
        if cantidad < 0:
            raise ValueError("la cura no puede ser negativa")
        if self.estaViva():
            self.__salud = min(self.__saludMaxima, self.__salud +
    cantidad)
```

Código 7. Encapsulamiento de `__salud`: sin setter y con validación.

Abstracción. EntidadJuego, Nivel, Tormenta y EstadoMarciano heredan de ABC y declaran sus métodos obligatorios con @abstractmethod, actuando como contratos que las subclases deben cumplir. Los parámetros y retornos están tipados con el módulo typing.

Uso del objeto en el juego. El Explorador es el objeto central: el jugador lo mueve con el teclado, él recolecta minerales (recolectar), construye refugios (construirBase) y recibe daño o curación de tormentas, criaturas, bases y marcianos aliados, siempre a través de los métodos públicos de EntidadJuego.

d. Patrones de diseño

Factory Method (TormentaFactory)

Aísla la creación de tormentas. Usa un registro privado (diccionario) que relaciona un identificador con una clase. Se pueden registrar nuevas tormentas sin modificar la fábrica, respetando el `principio` Open/Closed. Nivel solo pide una tormenta por nombre.

```
    class TormentaFactory:
        """Factory Method con registro dinámico: se registran tipos sin
    tocar esta clase (Open/Closed)."""

        def __init__(self) -> None:
            self.__registroTormentas: Dict[str, Type[Tormenta]] = {}

        def registrarTormenta(self, tipo: str, claseTormenta:
    Type[Tormenta]) -> None:
            if not tipo:
                raise ValueError("el tipo no puede estar vacío")
            self.__registroTormentas[tipo] = claseTormenta

        def crearTormenta(self, tipo: str) -> Tormenta:
```

```
            if tipo not in self.__registroTormentas:
                raise KeyError(f"Tipo de tormenta no registrado:
    {tipo}")
            return self.__registroTormentas[tipo]()

        def tiposDisponibles(self) -> List[str]:
            return list(self.__registroTormentas)
```

Código 8. Fábrica con registro dinámico.

```
    def main() -> None:
        # Registro dinámico (Open/Closed): una tormenta nueva = una
    línea aquí,
        # sin tocar TormentaFactory.
        fabrica = TormentaFactory()
        fabrica.registrarTormenta("radiacion", TormentaRadiacion)
        fabrica.registrarTormenta("meteoritos", TormentaMeteoritos)

        niveles = [NivelSupervivencia(fabrica), NivelDefensa(fabrica),
    NivelEscape(fabrica)]
        Juego(niveles).gameLoop()
```

Código 9. Registro de los tipos de tormenta y armado del juego en `main.py`.

State (Marciano)

La lógica condicional del marciano se extrajo a objetos de estado. Marciano no contiene if sobre su comportamiento: delega en estadoActual.ejecutarAccion(self, dt). Un marciano asustado huye del explorador; si este le ofrece un mineral, pasa a estado aliado, lo sigue y le cura vida.

```
    class MarcianoAsustado(EstadoMarciano):
        DISTANCIA_PANICO: float = 220.0

        def ejecutarAccion(self, contexto: Marciano, dt: float) -> None:
            objetivo = contexto.getObjetivo()
            if contexto.distanciaA(objetivo) < self.DISTANCIA_PANICO:
                contexto.alejarseDe(objetivo, dt, 1.8)
            else:
                contexto.deambular(dt)

        def colorCuerpo(self) -> Color:
            return (130, 205, 90)

        def aceptaOfrenda(self) -> bool:
            return True

    # Marciano (contexto)
    def cambiarEstado(self, nuevoEstado: EstadoMarciano) -> None:
        self.__estadoActual = nuevoEstado
    def update(self, dt: float) -> None:
```

```
        super().update(dt)
        self.__estadoActual.ejecutarAccion(self, dt)   # delegación al
    estado
```

Código 10. Patrón State: estado concreto y delegación desde el contexto.

V. Implementación de Resultados Parciales

a. Interfaces

El juego corre en una ventana gráfica de 960×640 píxeles con Pygame. Las figuras siguientes muestran el menú y los tres niveles jugables, con su HUD (salud, minerales, objetivo y puntaje).



Figura 2. Menú principal y controles.



Figura 3. Nivel 1: Supervivencia (radiación activa).

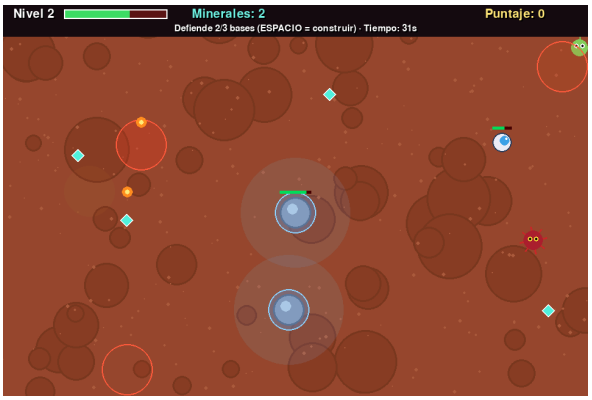


Figura 4. Nivel 2: Defensa de bases (meteoritos).

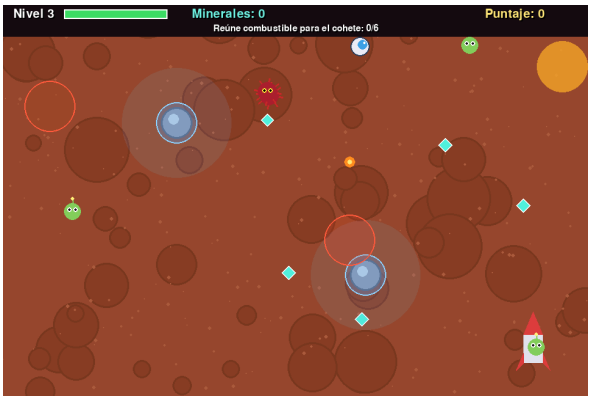


Figura 5. Nivel 3: Escape (cohete en la esquina).

b. Funcionalidades implementadas

Tabla 4. Estado de las funcionalidades

Funcionalidad	Clases involucradas	Estado
Menú, fases del juego, HUD y puntaje	`Juego`	Implementado

Funcionalidad	Clases involucradas	Estado
Movimiento del explorador (WASD / flechas)	`Explorador`	Implementado
Recolección de minerales y combustible	`Explorador, Mineral`	Implementado
Construcción de bases con escudo y zona de refugio	`Explorador, Base`	Implementado
Marcianos con estados asustado y aliado	`Marciano, EstadoMarciano`	Implementado
Criaturas hostiles que persiguen y atacan	`CriaturaMarciana`	Implementado
Tormentas de radiación y meteoritos creadas con la fábrica	`TormentaFactory, Tormenta y subclases`	Implementado
Tres niveles con objetivo y condición de victoria propios	`Nivel` y subclases	Implementado
Pantallas de victoria, derrota y reintento	`Juego`	Implementado
Tabla de puntuaciones en servicio web (APIClient)	APIClient	Pendiente
Sprites definitivos, sonido y balanceo fino	—	Pendiente

Verificación. Se ejecutaron partidas simuladas sin interfaz gráfica para comprobar que los tres niveles cargan, se actualizan sin errores y pueden completarse, y se revisó el código para confirmar que no se usan condicionales de tipo (isinstance).

c. Uso de Git durante el desarrollo

Para organizar el desarrollo del proyecto Misión en Marte, el equipo utiliza Git como sistema de control de versiones y GitHub como plataforma para almacenar el código fuente, la documentación y los avances del proyecto.

El repositorio oficial del equipo es: POO_Group_03.

Durante esta etapa parcial, el repositorio permite centralizar los archivos del proyecto, mantener un registro de los cambios realizados y facilitar la colaboración entre los

integrantes. Se han incorporado la documentación inicial y la estructura de carpetas que servirá de base para implementar el videojuego en Python.

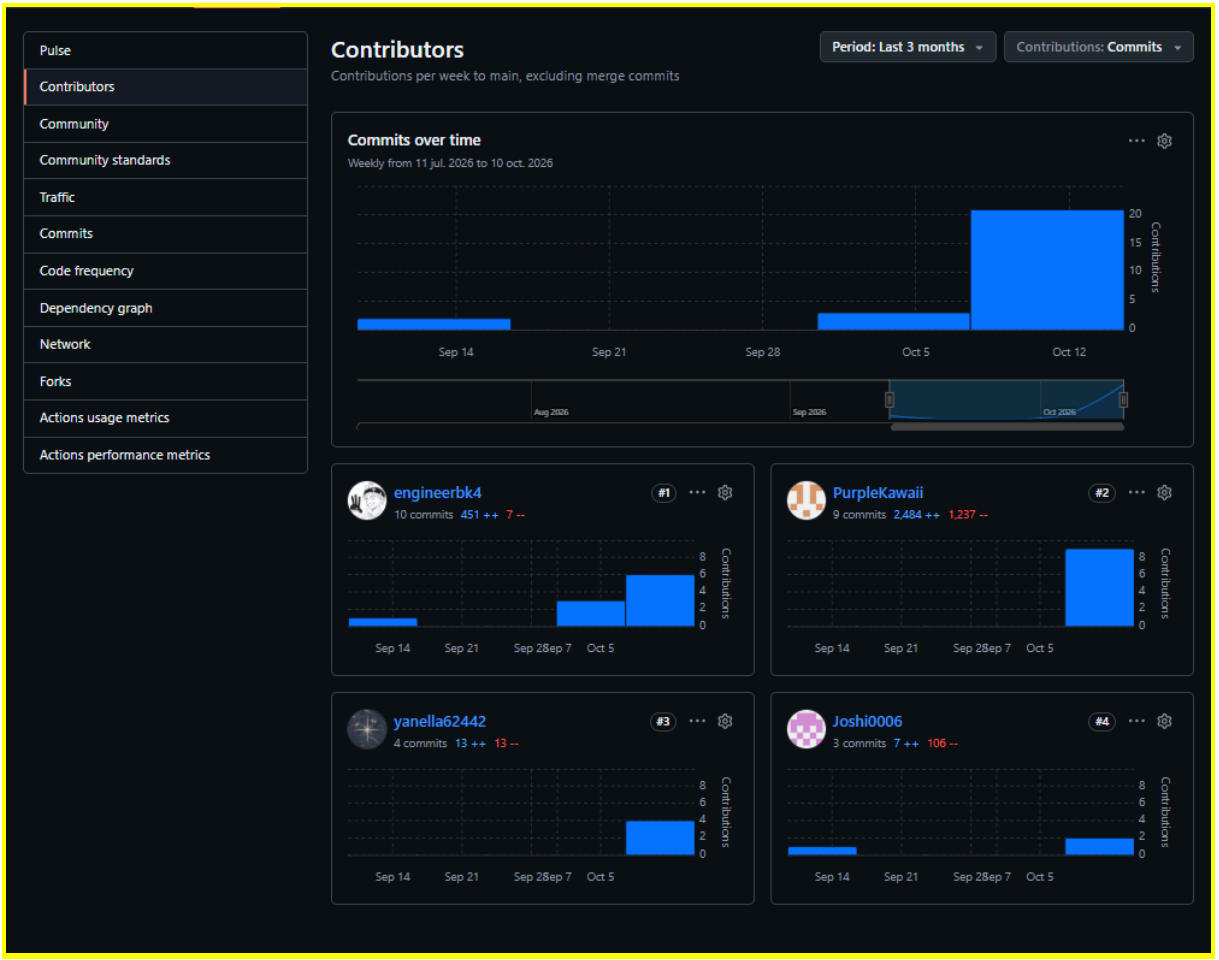
Los cambios se registran mediante *commits* con mensajes descriptivos que permiten identificar el propósito de cada modificación. Antes de integrar nuevos cambios, se debe comprobar que la versión local esté actualizada y revisar posibles conflictos entre las modificaciones de los integrantes.

La participación de cada integrante se documentará mediante el historial de commits del repositorio. Asimismo, se incluirán capturas de evidencia que permitan verificar los cambios realizados y el avance colaborativo del equipo.

Tabla 5. Aporte de cada integrante en el repositorio

Integrante	**Aporte principal**	**Commits**
Bances Panaque, Jose Junior	Diseño y revisión de la arquitectura orientada a objetos.	Revisar el UML, las relaciones entre clases y su correspondencia con la arquitectura del código.
Barzola Reategui, Josué Obed	Desarrollo del videojuego	Implementar clases y funcionalidades en Python, de acuerdo con las tareas que asuma.
Cerdan Dongo, Rebeca Milagros	Integración y documentación técnica	Apoyar en la organización del código, las pruebas y la documentación del avance.

Integrante	Aporte principal	Commits
Salcedo Siguas, Yanella Stefhany	Diseño visual e interfaz	Proponer la estética del videojuego, la temática de Marte, los recursos gráficos y la presentación visual.



VI. Conclusiones

Logros alcanzados

- Se diseñó e implementó una arquitectura de 18 clases coherente con el diagrama UML.
- Se aplicaron los seis pilares de la POO y los patrones Factory Method y State.

- El juego es jugable: tiene menú, tres niveles con objetivos distintos, enemigos, tormentas y pantallas de victoria y derrota.
- El comportamiento del juego se resuelve por polimorfismo, sin condicionales de tipo.

Aprendizajes clave y dificultades

- **Encapsulamiento frente a herencia:** con name mangling las subclases no pueden leer __salud; hubo que ofrecer getters y métodos de comportamiento, lo que mejoró el control del estado.
- **Polimorfismo real:** evitar isinstance() obligó a diseñar ganchos como esAfectablePorTormenta() y a mantener las reglas de cada nivel en sus subclases.
- **Diseño y código a la par:** al implementar fue necesario ajustar el UML inicial (por ejemplo, la salud pasó a ser privada y se añadieron métodos de apoyo), lo que mostró la importancia de mantenerlos sincronizados.
- **Trabajo en equipo con Git:** durante la integración con Git surgieron conflictos menores al fusionar ramas independientes y desajustes de rutas en el proyecto; se resolvieron estableciendo una disciplina estricta de *Pull Requests* y revisiones cruzadas.

Trabajo siguiente

Integrar APIClient para la tabla de puntuaciones, manteniendo el motor del juego independiente de la red; reemplazar las formas geométricas por sprites; incorporar sonido; y balancear la dificultad de los niveles.

VII. Anexos

Anexo A. Fragmentos de código relevantes

A.1 Game Loop (`Juego.gameLoop`)

```
    def gameLoop(self) -> None:
        while self.__corriendo:
            dt = min(self.__reloj.tick(config.FPS) / 1000.0, 0.05)
            self.__procesarEventos()
            self.__actualizar(dt)
            self.__dibujar()
        pygame.quit()
```

Código 11. Bucle principal: eventos, actualización y dibujo con `dt` constante.

A.2 Eliminación de referencias a objetos muertos

```
def __limpiarMuertos(self) -> None:
    """Elimina las referencias a objetos muertos para evitar fugas lógicas de
memoria."""
    self.__entidades = [e for e in self.__entidades if e.estaViva()]
    self._minerales[:] = [e for e in self._minerales if e.estaViva()]
    self._bases[:] = [e for e in self._bases if e.estaViva()]
    self._marcianos[:] = [e for e in self._marcianos if e.estaViva()]
    self._criaturas[:] = [e for e in self._criaturas if e.estaViva()]
```

Código 12. Limpieza de referencias en `Nivel`.

A.3 Escudo de la base (`Base.recibirDano`)

```
def recibirDano(self, cantidad: int) -> None:
    if self.__escudoActivo:
        self.__energiaEscudo -= cantidad
        if self.__energiaEscudo <= 0:
            self.__escudoActivo = False
            self.__recarga = 6.0
        cantidad -= cantidad // 2  # el escudo absorbe la mitad
    super().recibirDano(cantidad)
```

Código 13. Sobrescritura que usa `super()` para reutilizar la lógica base.

A.4 Efecto de la tormenta de radiación

```
def aplicarEfecto(self, entidades: List[EntidadJuego]) -> None:
    if self.haTerminado():
        return
    for e in entidades:
        dentro = math.hypot(e.getX() - self.__x, e.getY() - self.__y) <=
self.__radioEfecto
        if dentro and e.esAfectablePorTormenta():
            self.drenarVida(e)
```

Código 14. Aplicación polimórfica del efecto sobre la lista de entidades.

```
def drenarVida(self, e: EntidadJuego) -> None:
    acumulado = self.__acumulado.get(e.getId(), 0.0) + self._dano * self.__ultimoDt
    puntos = int(acumulado)
    if puntos >= 1:
        e.recibirDano(puntos)
        acumulado -= puntos
    self.__acumulado[e.getId()] = acumulado
```

Código 15. Daño por segundo acumulado por entidad.

A.5 Interfaz del patrón State

```
class EstadoMarciano(ABC):
    """Interfaz del patrón State: cada estado define su propia acción."""

    @abstractmethod
    def ejecutarAccion(self, contexto: Marciano, dt: float) -> None:
        ...

    @abstractmethod
    def colorCuerpo(self) -> Color:
        ...

    @abstractmethod
    def aceptaOfrenda(self) -> bool:
        …
```

Código 16. Contrato abstracto de los estados del marciano.